
Item 2 - Practice - Tertiary

Reminders

Typing text responses

Where you are instructed to type responses, use the files specified in the question text.

The margin characters, '>', indicate the maximum number of statements you can type.

Type at least nearly as many points as there are '>' characters to maximize your score.

Aim to keep statements brief and direct; prefer bullet points to longer sentences.

Case-sensitive by default

Unless stated otherwise, assume all instructions/examples are case-sensitive.

This means that 'A' is not the same as 'a', for example.

This can be relevant when referring to elements of source code or design parameters.

Text-Editing tools

You can open a text file in nano/vim by typing `nano <file>/vim <file>` in the terminal.

To open a file in kate via the terminal, type: `kate <file>` (this will consume the tab).

You can also open it by using the graphical application launcher in the corner.

Writing code

Aim to write the shortest amount of code which fulfills the specification.

Your code may be marked for readability and style.

Read the commented header of the response file for advice.

This is a practice paper

Some marking schemes are visible to inform your approach and response.

This item can only give you an idea of the format and tone of the actual item.

You should complete your class-based practice tasks and request feedback (if necessary).

Reminders (Technology-Specific)

Basic Bash Commands Cheat Sheet

- `pwd` – Print the current working directory
- `ls` – List files and directories in the current location
- `cd <dir>` – Change to directory <dir>
- `mkdir <dir>` – Create a new directory named <dir>
- `rm <file>` – Remove a file
- `cp <src> <dest>` – Copy a file from <src> to <dest>
- `mv <src> <dest>` – Move or rename a file from <src> to <dest>
- `cat <file>` – Display the contents of a file
- `clear` – Clear the terminal screen

Executing Python scripts

To run your Python script, `your_file.py`, from the terminal:

1. Navigate to the directory containing your `your_file.py`
2. Execute `python your_file.py`

Canberra College IT Faculty

Canberra College IT Faculty

Jump To

Q1 (2 parts)	4
Q2 (4 parts)	6
Q3 (4 parts)	8
Q4 (2 parts)	11
Q5 (1 part)	12
Q6 (2 parts)	13

Canberra College IT Faculty
Canberra College IT Faculty

*The question of whether computers can think is like the
question of whether submarines can swim.*

— Edsger W. Dijkstra

*If you can't explain it simply, you don't understand it well
enough.*

— Albert Einstein

Q1 (2 parts)



Canberra College IT Faculty

a) Imagine you are building BadgeMint, an app that checks whether a workshop badge can be issued from one typed line. Write a Python program which:

- asks the user for one line containing a year and a session number separated by a space
- uses try and except to handle input that cannot be converted into two integers
- prints Accepted if the year is from 2020 to 2026 inclusive and the session number is from 1 to 4 inclusive otherwise prints Rejected

The following are **example** interactions.

User/assessor input is shown in **bold**.

Your program may be tested with other inputs not shown here.

You may assume that only inputs of the type shown will be used.

Enter year and session: **2024 3**

Accepted

Enter year and session: **2027 2**

Rejected

Enter year and session: **word 2**

Rejected

Save your source code in `q1a_response.py`.

Canberra College IT Faculty

b) BadgeMint stores attendee names and optional contact details for short campus events. In 3 to 5 sentences, explain two ethical responsibilities the team must address before coordinators can export these details for wider staff use.

Save your response in `q1_response.txt`.

Canberra College IT Faculty

Q2 (4 parts)



a) Now you are building ShelfSnap, an app for quick exhibit cards. Write a Python program which:

- opens a file named `raw_cards.csv`
- reads each line from the file
- treats each non blank line as `title,category`
- removes leading and trailing whitespace around both values and ignores lines that do not contain exactly one comma
- writes the cleaned pairs to `cards_cleaned.csv` in the form `title|category`
- prints the number of cleaned pairs written

Assume that this file is in the same directory as your program.

Contents of the **input file**:

```
mask , culture
```

```
poster, learning
```

```
bad,line,extra
```

```
map , travel
```

This file must be created and saved in the same directory as your program.

Required contents of the **output file**:

```
mask|culture
```

```
poster|learning
```

```
map|travel
```

The following must be printed to the terminal after execution of your program.

Required Output:

```
3
```

Save your source code in `q2a_response.py`.

b) ShelfSnap has grown. The script has gradually become one long file where the reading, cleaning, formatting, and export steps are all mixed together. A new student team is planning to refactor the code so that it is easier to understand and maintain in future semesters. In 3 to 5 sentences, propose two review checkpoints you would use to judge whether a refactor actually improved maintainability for the next student team.

Save your response in `q2_response.txt`.

c) ShelfSnap will be used by librarians, student workers, and visiting volunteers. In 3 to 5 sentences, propose a short governance note that would improve ethical handling of contributor notes and explain why it would help routine work.

Save your response in `q2_response.txt`.

d) In ShelfSnap, volunteers upload raw card text that is later cleaned by the script in part (a). For example, extra spaces may be removed and valid `title , category` pairs are written to `cards_cleaned.csv`. Sometimes librarians later discover that a title or category was corrected during this cleaning process. A new volunteer asks why ShelfSnap should record who cleaned a card entry and when the change happened. In 3 to 4 sentences, explain how a brief provenance note would help the team review and correct catalogue records later.

Save your response in `q2_response.txt`.

**Q3 (4 parts)**

a) PlaylistPick is an app that chooses a featured song from a playlist. Write a Python program which:

- asks the user for one line containing an integer seed and an integer song count separated by a comma
- uses `random.seed` with the entered seed
- uses `random.randint` to choose a featured song number from 1 to the song count inclusive
- prints the result in the form `Featured song: number`

The following are **example** interactions.

User/assessor input is shown in **bold**.

Your program may be tested with other inputs not shown here.

You may assume that only inputs of the type shown will be used.

Enter seed, song_count: **4,10**

Featured song: 4

Enter seed, song_count: **2,5**

Featured song: 1

Save your source code in `q3a_response.py`.

b) QueueLens is an app that receives a growing stream of check in events during festivals. In 3 to 5 sentences, explain how one architectural choice could help the event pipeline scale as traffic grows from hundreds to tens of thousands of events per hour.

Save your response in `q3_response.txt`.

c) ClipWeave is an app that builds a long transcript summary by repeatedly joining extra text inside nested loops. In 3 to 5 sentences, explain why this pattern may be inefficient and describe one more efficient alternative.

Save your response in `q3_response.txt`.

d) RouteBoard is a small micro-app used by event volunteers to record walking routes around a venue. Each route entry stores the path name, the distance in metres, and the time taken to walk it. The team believes the app may need performance optimisation later.

Note: ↪ is used to indicate long lines and is not part of the original code.

```
# RouteBoard example: records simple walking route entries
# and measures how long the processing takes.

import time

def main():
    routes = [
        ("North Gate Loop", 420, 360),
        ("Stage Circle", 310, 240),
        ("Food Court Path", 280, 210),
        ("River Walk", 650, 520),
        ("Main Arena Loop", 500, 400),
    ]
```

```
start = time.perf_counter()

total_distance = 0
total_time = 0

for name, distance, walk_time in routes:
    total_distance += distance
    total_time += walk_time

    speed = distance / walk_time
    print(name, "speed:", round(speed, 2), "m/s")

end = time.perf_counter()

print()
print("Total distance:", total_distance, "metres")
print("Total time:", total_time, "seconds")
print("Elapsed processing time:", end - start)

if __name__ == "__main__":
    main()
```

Save your response in q3_response.txt.

Q4 (2 parts)



a) NameNest is an app that presents pronunciation clips, optional translations, and small profile images for student groups. In 4 to 6 sentences, analyse how these digital assets should be adapted for Australian First Nations cultural fit before wider campus release. You may consider issues such as the respectful use of names, language, recordings, images, consultation, and permissions. End with one specific review action.

Save your response in `q4_response.txt`.

b) MapMarker is an app that inherits old location labels from earlier campus systems, including building names, pathway markers, and place tags still shown in search and navigation results. In 3 to 5 sentences, explain one social or historical factor that shaped those labels and suggest one respectful adaptation for current users.

Canberra College IT Faculty

Save your response in `q4_response.txt`.

Canberra College IT Faculty

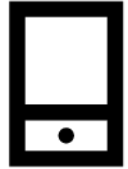
**Q5 (1 part)**

a) GuideGrid is an app that shows the same campus collection item in a tutor dashboard and in a public browse screen. In 4 to 6 sentences, analyse how the asset may need different framing or interface treatment across these two contexts given the following purposes of each:

- Tutor dashboard: preparing class examples and internal notes.
- Public browse screen: first time visitors exploring short descriptions.

Save your response in `q5_response.txt`.

Q6 (2 parts)



a) TokenTrail is an app that checks short share codes before a file can be previewed. Write a Python program which:

- asks the user for a code in the form letters digits letters such as AB12Z
- prints Valid if the code has exactly 5 characters and the pattern is 2 uppercase letters followed by 2 digits followed by 1 uppercase letter otherwise prints Invalid

The following are **example** interactions.

User/assessor input is shown in **bold**.

Your program may be tested with other inputs not shown here.

You may assume that only inputs of the type shown will be used.

Enter share code: **AB12Z**

Valid

Enter share code: **A912Z**

Invalid

Enter share code: **AB123**

Invalid

Save your source code in `q6a_response.py`.

Canberra College IT Faculty

Canberra College IT Faculty

b) In 3 to 4 sentences, explain one reason why a pattern based code rule like the one in TokenTrail can reduce accidental sharing mistakes, and one limitation it still has.

Save your response in `q6_response.txt`.

Canberra College IT Faculty

Canberra College IT Faculty